Python 30-Day Expected Solutions

30 days • test-driven development • Python basics to LeetCode foundations

How to Use This

Follow a strict TDD loop: write one failing test, implement the smallest passing solution, then refactor.

Use pytest for every challenge. Keep each day's code in its own folder with solution.py and test_solution.py.

Do not read the expected solution until you have made a real attempt.

LeetCode days are included twice a week to build pattern recognition gradually.

Suggested Folder Structure

```
python-practice/
  day01/
    solution.py
    test_solution.py
  day02/
    solution.py
    test_solution.py
  ...
```

Pytest Quick Start

```
pip install pytest
pytest
```

Each day in this solutions guide includes a reference implementation and short notes on why the solution works.

Day 1 — Swap Two Variables

Reference function: swap_values(a, b)

Problem Reminder

Write a function that takes two values and returns them in swapped order.

Reference Solution

```
def swap_values(a, b):
    return b, a
```

Reference Tests

```
from solution import swap_values

def test_swap_integers():
    assert swap_values(3, 5) == (5, 3)

def test_swap_strings():
    assert swap_values("a", "b") == ("b", "a")

def test_swap_same_values():
    assert swap_values(7, 7) == (7, 7)
```

Why This Works

This is a tiny challenge, but it sets up the habit of writing a test before implementation.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 2 — Even or Odd

Reference function: even_or_odd(n)

Problem Reminder

Return 'even' if the input integer is even, otherwise return 'odd'.

Reference Solution

```
def even_or_odd(n):
    return "even" if n % 2 == 0 else "odd"
```

Reference Tests

```
from solution import even_or_odd

def test_even_number():
    assert even_or_odd(10) == "even"

def test_odd_number():
    assert even_or_odd(7) == "odd"

def test_zero():
    assert even_or_odd(0) == "even"

def test_negative_odd():
    assert even_or_odd(-3) == "odd"
```

Why This Works

The `%` operator is central here. `n % 2 == 0` means there is no remainder when dividing by 2.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 3 — Sum From 1 to n

Reference function: sum_to_n(n)

Problem Reminder

Return the sum of all integers from 1 up to n. For this workbook, return 0 for non-positive numbers.

Reference Solution

```
def sum_to_n(n):
    if n <= 0:
        return 0
    return n * (n + 1) // 2
```

Reference Tests

```
from solution import sum_to_n

def test_sum_to_5():
    assert sum_to_n(5) == 15

def test_sum_to_1():
    assert sum_to_n(1) == 1

def test_sum_to_0():
    assert sum_to_n(0) == 0

def test_negative_returns_zero():
    assert sum_to_n(-3) == 0
```

Why This Works

A loop is perfectly fine for the first solution. The arithmetic formula is a good refactor once the behavior is covered by tests.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 4 — Palindrome Checker

Reference function: is_palindrome(text)

Problem Reminder

Return True if a string reads the same forward and backward, otherwise False.

Reference Solution

```
def is_palindrome(text):
    return text == text[::-1]
```

Reference Tests

```
from solution import is_palindrome

def test_level_is_palindrome():
    assert is_palindrome("level") is True

def test_hello_is_not_palindrome():
    assert is_palindrome("hello") is False

def test_single_character():
    assert is_palindrome("a") is True

def test_empty_string():
    assert is_palindrome("") is True
```

Why This Works

String slicing with `[::-1]` creates a reversed copy of the string.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 5 — Largest Number in a List

Reference function: find_max(numbers)

Problem Reminder

Return the largest number in a list. Raise ValueError for an empty list.

Reference Solution

```
def find_max(numbers):
    if not numbers:
        raise ValueError("numbers cannot be empty")
    current_max = numbers[0]
    for number in numbers[1:]:
        if number > current_max:
            current_max = number
    return current_max
```

Reference Tests

```
from solution import find_max
import pytest

def test_find_max_normal_list():
    assert find_max([3, 8, 2, 10, 4]) == 10

def test_find_max_single_item():
    assert find_max([7]) == 7

def test_find_max_negative_numbers():
    assert find_max([-5, -2, -9]) == -2

def test_find_max_empty_list():
    with pytest.raises(ValueError):
        find_max([])
```

Why This Works

Writing the manual loop teaches comparison logic. You can later compare it to Python's built-in `max()`.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 6 — LeetCode - Two Sum

Reference function: two_sum(nums, target)

Problem Reminder

Given a list of integers and a target value, return the indices of two numbers whose sum equals the target.

Reference Solution

```
def two_sum(nums, target):
    seen = {}
    for index, number in enumerate(nums):
        complement = target - number
        if complement in seen:
            return [seen[complement], index]
        seen[number] = index
```

Reference Tests

```
from solution import two_sum

def test_example_case():
    assert two_sum([2, 7, 11, 15], 9) == [0, 1]

def test_second_example():
    assert two_sum([3, 2, 4], 6) == [1, 2]

def test_duplicates():
    assert two_sum([3, 3], 6) == [0, 1]
```

Why This Works

The dictionary stores numbers already seen and their indices. That cuts the time complexity from $O(n^2)$ to $O(n)$.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 7 — Remove Duplicates While Preserving Order

Reference function: remove_duplicates(items)

Problem Reminder

Return a new list with duplicates removed while keeping the first occurrence of each item.

Reference Solution

```
def remove_duplicates(items):
    result = []
    for item in items:
        if item not in result:
            result.append(item)
    return result
```

Reference Tests

```
from solution import remove_duplicates

def test_remove_duplicates_normal():
    assert remove_duplicates([1, 2, 2, 3, 1, 4]) == [1, 2, 3, 4]

def test_no_duplicates():
    assert remove_duplicates([1, 2, 3]) == [1, 2, 3]

def test_empty_list():
    assert remove_duplicates([]) == []

def test_all_same():
    assert remove_duplicates([5, 5, 5]) == [5]
```

Why This Works

A set removes duplicates quickly, but it does not preserve order in the same way. This exercise is specifically about order-preserving behavior.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 8 — Maximum of Two Numbers

Reference function: max_of_two(a, b)

Problem Reminder

Return the larger of two numbers. If the numbers are equal, return that value.

Reference Solution

```
def max_of_two(a, b):
    return a if a >= b else b
```

Reference Tests

```
from solution import max_of_two

def test_first_is_larger():
    assert max_of_two(10, 5) == 10

def test_second_is_larger():
    assert max_of_two(2, 8) == 8

def test_equal_values():
    assert max_of_two(4, 4) == 4

def test_negative_values():
    assert max_of_two(-1, -5) == -1
```

Why This Works

This is a good place to practice writing tiny, precise tests.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 9 — Longest Word in a List

Reference function: longest_word(words)

Problem Reminder

Return the longest word in a list. If there is a tie, return the first longest word. Raise ValueError for an empty list.

Reference Solution

```
def longest_word(words):
    if not words:
        raise ValueError("words cannot be empty")
    longest = words[0]
    for word in words[1:]:
        if len(word) > len(longest):
            longest = word
    return longest
```

Reference Tests

```
from solution import longest_word
import pytest

def test_longest_word_normal():
    assert longest_word(["cat", "elephant", "dog"]) == "elephant"

def test_tie_returns_first():
    assert longest_word(["bat", "car", "sun"]) == "bat"

def test_single_word():
    assert longest_word(["hello"]) == "hello"

def test_empty_list():
    with pytest.raises(ValueError):
        longest_word([])
```

Why This Works

Using a manual loop makes the tie behavior explicit and easy to reason about.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 10 — Character Frequency Counter

Reference function: char_frequency(text)

Problem Reminder

Return a dictionary showing how many times each character appears in the input string.

Reference Solution

```
def char_frequency(text):
    counts = {}
    for char in text:
        counts[char] = counts.get(char, 0) + 1
    return counts
```

Reference Tests

```
from solution import char_frequency

def test_banana_counts():
    assert char_frequency("banana") == {"b": 1, "a": 3, "n": 2}

def test_empty_string():
    assert char_frequency("") == {}

def test_single_character():
    assert char_frequency("a") == {"a": 1}

def test_spaces_counted():
    assert char_frequency("a a") == {"a": 2, " ": 1}
```

Why This Works

This is a foundational dictionary exercise. The `.get()` method keeps the code compact and readable.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 11 — Common Elements in Two Lists

Reference function: common_elements(list1, list2)

Problem Reminder

Return a list of unique values that appear in both lists.

Reference Solution

```
def common_elements(list1, list2):
    set2 = set(list2)
    result = []
    seen = set()
    for item in list1:
        if item in set2 and item not in seen:
            result.append(item)
            seen.add(item)
    return result
```

Reference Tests

```
from solution import common_elements

def test_common_elements_normal():
    assert common_elements([1, 2, 3, 4], [3, 4, 5, 6]) == [3, 4]

def test_no_common_elements():
    assert common_elements([1, 2], [3, 4]) == []

def test_duplicates_in_input():
    assert common_elements([1, 1, 2, 3], [1, 1, 3]) == [1, 3]

def test_empty_input():
    assert common_elements([], [1, 2]) == []
```

Why This Works

A pure set intersection is shorter, but this version preserves the order of the first list.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 12 — Student Record Lookup

Reference function: get_student_mark(records, name)

Problem Reminder

Given a dictionary mapping student names to marks, return the student's mark if found, otherwise return 'Not found'.

Reference Solution

```
def get_student_mark(records, name):
    return records.get(name, "Not found")
```

Reference Tests

```
from solution import get_student_mark

def test_student_exists():
    assert get_student_mark({"Tom": 70, "Sara": 85}, "Sara") == 85

def test_student_missing():
    assert get_student_mark({"Tom": 70}, "Mike") == "Not found"

def test_empty_records():
    assert get_student_mark({}, "Tom") == "Not found"
```

Why This Works

This is an ideal use case for `dict.get()`.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 13 — LeetCode - Valid Anagram

Reference function: is_anagram(s, t)

Problem Reminder

Return True if the two strings are anagrams of each other, otherwise return False.

Reference Solution

```
def is_anagram(s, t):
    if len(s) != len(t):
        return False

    counts = {}
    for char in s:
        counts[char] = counts.get(char, 0) + 1

    for char in t:
        if char not in counts:
            return False
        counts[char] -= 1
        if counts[char] < 0:
            return False

    return True
```

Reference Tests

```
from solution import is_anagram

def test_valid_anagram():
    assert is_anagram("anagram", "nagaram") is True

def test_not_anagram():
    assert is_anagram("rat", "car") is False

def test_empty_strings():
    assert is_anagram("", "") is True

def test_different_lengths():
    assert is_anagram("a", "ab") is False
```

Why This Works

Sorting is the easiest first version. Counting characters is a great dictionary pattern to practice.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 14 — Word Frequency in a Sentence

Reference function: word_frequency(sentence)

Problem Reminder

Return a dictionary with the number of times each word appears in a sentence.

Reference Solution

```
def word_frequency(sentence):
    counts = {}
    for word in sentence.split():
        counts[word] = counts.get(word, 0) + 1
    return counts
```

Reference Tests

```
from solution import word_frequency

def test_sentence_word_count():
    assert word_frequency("the cat and the dog") == {
        "the": 2, "cat": 1, "and": 1, "dog": 1
    }

def test_empty_sentence():
    assert word_frequency("") == {}

def test_single_word():
    assert word_frequency("hello") == {"hello": 1}
```

Why This Works

This is the word-based version of Day 10's character-count problem.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 15 — Count Even Numbers in a List

Reference function: count_evens(numbers)

Problem Reminder

Return how many even numbers exist in the input list.

Reference Solution

```
def count_evens(numbers):
    count = 0
    for number in numbers:
        if number % 2 == 0:
            count += 1
    return count
```

Reference Tests

```
from solution import count_evens

def test_count_evens_normal():
    assert count_evens([1, 2, 3, 4, 6]) == 3

def test_no_evens():
    assert count_evens([1, 3, 5]) == 0

def test_all_evens():
    assert count_evens([2, 4, 6]) == 3

def test_empty_list():
    assert count_evens([]) == 0
```

Why This Works

This problem strengthens counting logic and loop control.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 16 — Move Zeroes

Reference function: move_zeroes(nums)

Problem Reminder

Return a list where all zeroes are moved to the end while preserving the order of non-zero elements.

Reference Solution

```
def move_zeroes(nums):
    non_zeroes = [num for num in nums if num != 0]
    zero_count = nums.count(0)
    return non_zeroes + [0] * zero_count
```

Reference Tests

```
from solution import move_zeroes

def test_move_zeroes_example():
    assert move_zeroes([0, 1, 0, 3, 12]) == [1, 3, 12, 0, 0]

def test_no_zeroes():
    assert move_zeroes([1, 2, 3]) == [1, 2, 3]

def test_all_zeroes():
    assert move_zeroes([0, 0, 0]) == [0, 0, 0]

def test_empty():
    assert move_zeroes([]) == []
```

Why This Works

LeetCode often asks for in-place modification, but for this workbook a returned list keeps the function simpler.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 17 — Sort Words by Length

Reference function: sort_by_length(words)

Problem Reminder

Return the words sorted from shortest to longest. If lengths are equal, keep original order.

Reference Solution

```
def sort_by_length(words):
    return sorted(words, key=len)
```

Reference Tests

```
from solution import sort_by_length

def test_sort_by_length_normal():
    assert sort_by_length(["apple", "kiwi", "banana"]) == ["kiwi", "apple", "banana"]

def test_empty_list():
    assert sort_by_length([]) == []

def test_same_length_words():
    assert sort_by_length(["bat", "cat", "dog"]) == ["bat", "cat", "dog"]
```

Why This Works

Python's sort is stable, so equal-length items keep their original order.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 18 — Square Numbers

Reference function: square_numbers(numbers)

Problem Reminder

Return a new list containing the square of each number in the input list.

Reference Solution

```
def square_numbers(numbers):
    return [number * number for number in numbers]
```

Reference Tests

```
from solution import square_numbers

def test_square_numbers_normal():
    assert square_numbers([1, 2, 3, 4]) == [1, 4, 9, 16]

def test_empty_list():
    assert square_numbers([]) == []

def test_negative_numbers():
    assert square_numbers([-2, -1, 0, 3]) == [4, 1, 0, 9]
```

Why This Works

This is a clean introduction to list comprehensions.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 19 — Safe Division

Reference function: safe_divide(a, b)

Problem Reminder

Return the result of dividing a by b. If b is zero, return 'Cannot divide by zero'.

Reference Solution

```
def safe_divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Cannot divide by zero"
```

Reference Tests

```
from solution import safe_divide

def test_normal_division():
    assert safe_divide(10, 2) == 5.0

def test_division_by_zero():
    assert safe_divide(10, 0) == "Cannot divide by zero"

def test_negative_division():
    assert safe_divide(-9, 3) == -3.0
```

Why This Works

Catching the specific exception is better than using a blanket `except:`.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 20 — LeetCode - Valid Parentheses

Reference function: is_valid_parentheses(s)

Problem Reminder

Given a string containing only brackets `()[]{}`, return True if every opening bracket is closed in the correct order.

Reference Solution

```
def is_valid_parentheses(s):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []

    for char in s:
        if char in pairs.values():
            stack.append(char)
        elif char in pairs:
            if not stack or stack.pop() != pairs[char]:
                return False

    return not stack
```

Reference Tests

```
from solution import is_valid_parentheses

def test_valid_case():
    assert is_valid_parentheses("()[]{}") is True

def test_invalid_case():
    assert is_valid_parentheses("(]") is False

def test_nested_valid_case():
    assert is_valid_parentheses("({[]})") is True

def test_unclosed_opening():
    assert is_valid_parentheses("(") is False
```

Why This Works

This is a classic stack problem and a good interview pattern to know.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 21 — LeetCode - Best Time to Buy and Sell Stock

Reference function: max_profit(prices)

Problem Reminder

Given daily stock prices, return the maximum profit from exactly one buy and one sell. If no profit is possible, return 0.

Reference Solution

```
def max_profit(prices):
    min_price = float("inf")
    best_profit = 0

    for price in prices:
        if price < min_price:
            min_price = price
        profit = price - min_price
        if profit > best_profit:
            best_profit = profit

    return best_profit
```

Reference Tests

```
from solution import max_profit

def test_example_case():
    assert max_profit([7, 1, 5, 3, 6, 4]) == 5

def test_no_profit_possible():
    assert max_profit([7, 6, 4, 3, 1]) == 0

def test_single_day():
    assert max_profit([5]) == 0
```

Why This Works

This is an excellent one-pass interview problem for practicing state tracking.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 22 — Factorial

Reference function: factorial(n)

Problem Reminder

Return the factorial of a non-negative integer n. Raise ValueError for negative input.

Reference Solution

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be non-negative")

    result = 1
    for value in range(2, n + 1):
        result *= value
    return result
```

Reference Tests

```
from solution import factorial
import pytest

def test_factorial_5():
    assert factorial(5) == 120

def test_factorial_0():
    assert factorial(0) == 1

def test_factorial_1():
    assert factorial(1) == 1

def test_factorial_negative():
    with pytest.raises(ValueError):
        factorial(-1)
```

Why This Works

Iterative factorial is easier to debug. Recursive factorial is a useful follow-up exercise.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 23 — Linear Search

Reference function: linear_search(items, target)

Problem Reminder

Return the index of the target if present in the list, otherwise return -1.

Reference Solution

```
def linear_search(items, target):
    for index, value in enumerate(items):
        if value == target:
            return index
    return -1
```

Reference Tests

```
from solution import linear_search

def test_target_found():
    assert linear_search([4, 2, 7, 9], 7) == 2

def test_target_not_found():
    assert linear_search([4, 2, 7, 9], 5) == -1

def test_empty_list():
    assert linear_search([], 1) == -1
```

Why This Works

This is one of the most fundamental algorithmic patterns.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 24 — Queue Simulation

Reference function: SimpleQueue.enqueue / SimpleQueue.dequeue

Problem Reminder

Implement a simple queue class with enqueue and dequeue methods. Dequeue should raise IndexError if the queue is empty.

Reference Solution

```
class SimpleQueue:
    def __init__(self):
        self._items = []

    def enqueue(self, item):
        self._items.append(item)

    def dequeue(self):
        if not self._items:
            raise IndexError("queue is empty")
        return self._items.pop(0)
```

Reference Tests

```
from solution import SimpleQueue
import pytest

def test_queue_enqueue_dequeue():
    q = SimpleQueue()
    q.enqueue(1)
    q.enqueue(2)
    assert q.dequeue() == 1
    assert q.dequeue() == 2

def test_queue_empty_dequeue():
    q = SimpleQueue()
    with pytest.raises(IndexError):
        q.dequeue()
```

Why This Works

Using a list is fine for learning. Later, `collections.deque` is the better queue structure.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 25 — First Non-Repeating Character

Reference function: first_unique_char(text)

Problem Reminder

Return the first character that appears exactly once in the string. Return None if every character repeats.

Reference Solution

```
def first_unique_char(text):
    counts = {}
    for char in text:
        counts[char] = counts.get(char, 0) + 1

    for char in text:
        if counts[char] == 1:
            return char

    return None
```

Reference Tests

```
from solution import first_unique_char

def test_first_unique_exists():
    assert first_unique_char("swiss") == "w"

def test_all_repeating():
    assert first_unique_char("aabbcc") is None

def test_empty_string():
    assert first_unique_char("") is None

def test_single_char():
    assert first_unique_char("z") == "z"
```

Why This Works

The first pass counts. The second pass preserves original order when checking uniqueness.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 26 — FizzBuzz Variant

Reference function: fizzbuzz(n)

Problem Reminder

Return a list of strings from 1 to n using these rules: multiples of 3 become 'Fizz', multiples of 5 become 'Buzz', multiples of both become 'FizzBuzz'.

Reference Solution

```
def fizzbuzz(n):
    result = []
    for number in range(1, n + 1):
        if number % 15 == 0:
            result.append("FizzBuzz")
        elif number % 3 == 0:
            result.append("Fizz")
        elif number % 5 == 0:
            result.append("Buzz")
        else:
            result.append(str(number))
    return result
```

Reference Tests

```
from solution import fizzbuzz

def test_fizzbuzz_5():
    assert fizzbuzz(5) == ["1", "2", "Fizz", "4", "Buzz"]

def test_fizzbuzz_15_last_value():
    assert fizzbuzz(15)[-1] == "FizzBuzz"

def test_fizzbuzz_1():
    assert fizzbuzz(1) == ["1"]

def test_fizzbuzz_0():
    assert fizzbuzz(0) == []
```

Why This Works

FizzBuzz looks simple, but it is useful practice for branch ordering and test completeness.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 27 — LeetCode - Binary Search

Reference function: binary_search(nums, target)

Problem Reminder

Search for a target in a sorted list and return its index, or -1 if not found.

Reference Solution

```
def binary_search(nums, target):
    left = 0
    right = len(nums) - 1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
```

Reference Tests

```
from solution import binary_search

def test_target_found():
    assert binary_search([-1, 0, 3, 5, 9, 12], 9) == 4

def test_target_not_found():
    assert binary_search([-1, 0, 3, 5, 9, 12], 2) == -1

def test_single_item_found():
    assert binary_search([5], 5) == 0

def test_empty_list():
    assert binary_search([], 1) == -1
```

Why This Works

Binary search requires sorted input. That precondition is part of the problem contract.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 28 — Reverse a String

Reference function: reverse_string(text)

Problem Reminder

Return a reversed version of the input string.

Reference Solution

```
def reverse_string(text):
    reversed_chars = []
    for index in range(len(text) - 1, -1, -1):
        reversed_chars.append(text[index])
    return "".join(reversed_chars)
```

Reference Tests

```
from solution import reverse_string

def test_reverse_normal_word():
    assert reverse_string("python") == "nohtyp"

def test_reverse_empty():
    assert reverse_string("") == ""

def test_reverse_single_char():
    assert reverse_string("a") == "a"

def test_reverse_with_spaces():
    assert reverse_string("ab cd") == "dc ba"
```

Why This Works

A loop-based version helps you understand indexing before using shortcuts like `text[::-1]`.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 29 — Intersection of Two Arrays

Reference function: intersection(nums1, nums2)

Problem Reminder

Return the values common to both arrays, with each value appearing only once in the result.

Reference Solution

```
def intersection(nums1, nums2):
    return list(set(nums1) & set(nums2))
```

Reference Tests

```
from solution import intersection

def test_intersection_normal():
    assert intersection([1, 2, 2, 1], [2, 2]) == [2]

def test_multiple_common_values():
    assert sorted(intersection([4, 9, 5], [9, 4, 9, 8, 4])) == [4, 9]

def test_no_intersection():
    assert intersection([1, 2], [3, 4]) == []

def test_empty_input():
    assert intersection([], [1, 2]) == []
```

Why This Works

This is a good introduction to set intersection.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.

Day 30 — LeetCode - Longest Common Prefix

Reference function: longest_common_prefix(strs)

Problem Reminder

Return the longest starting substring shared by all strings in the list. Return an empty string if there is none.

Reference Solution

```
def longest_common_prefix(strs):
    if not strs:
        return ""

    prefix = strs[0]
    for word in strs[1:]:
        while not word.startswith(prefix):
            prefix = prefix[:-1]
            if prefix == "":
                return ""
    return prefix
```

Reference Tests

```
from solution import longest_common_prefix

def test_common_prefix_exists():
    assert longest_common_prefix(["flower", "flow", "flight"]) == "fl"

def test_no_common_prefix():
    assert longest_common_prefix(["dog", "racecar", "car"]) == ""

def test_single_word():
    assert longest_common_prefix(["alone"]) == "alone"

def test_empty_list():
    assert longest_common_prefix([]) == ""
```

Why This Works

This is a strong final checkpoint because it combines loops, conditions, and string methods.

Try This After Solving

Refactor your own version. Compare readability, edge-case handling, and test coverage to this reference implementation.